# MEJARC PLATFORM

Integration Specifications & Developer Guidelines

TECHNICAL REFERENCE

## 1. ARCHITECTURE & FRONTEND DEPLOYMENT OPTIONS

Mejarc follows an API-first, decoupled architecture allowing versatile frontend integration patterns:

### React / Next.js Components (Recommended)

Modular component integration consuming REST API directly. Preferred tech stack is React / Next.js with TypeScript and Tailwind/Vanilla CSS.

### Micro-Frontends

Supported via Webpack/Vite Module Federation or single-spa wrappers for segregated modules (e.g. Agent Portal, Admin Console, Marketplace).

### iFrames & Embedded Widgets

Supported for standalone checkout or verification widgets. Communicates with parent host application securely via `window.postMessage`.

### Custom HTTP Clients

Standard headless integration using any modern web or mobile runtime (Vue, Svelte, Flutter, React Native, Swift, Kotlin).

## 2. AUTHENTICATION & SESSION CONTEXT

Authentication is handled via stateful / two-factor authenticated **JSON Web Tokens (JWT)**.

- **Authentication Handshake:**
  1. `POST /user/login` (or `/agent/login` / `/admin/login`) sends credentials and initiates email OTP verification.
  2. `POST /user/verify-login-token` validates the OTP code and returns a signed `accessToken` plus profile details.
- **Header Authorization:** All protected requests must supply the Bearer token:

```
Authorization: Bearer <accessToken>
```

- **Context Sharing with External Extensions:**
  - **Same-domain micro-frontends:** Context stored in secure `HttpOnly` cookies or encrypted session storage.
  - **Cross-origin iFrame extensions:** Strict handshake via `window.postMessage` validating `event.origin` to exchange short-lived extension tokens.
  - **Role-Based Access Control (RBAC):** Validated server-side via token payload roles (`User`, `Agent`, `Admin`, `SuperAdmin`).

## 3. PLATFORM APIS & SWAGGER DOCUMENTATION

Mejarc's backend is powered by **NestJS (TypeScript)** with PostgreSQL and TypeORM.

- **Swagger / OpenAPI Interactive UI:** Available at `/api` (e.g., `https://api-dev.mejarc.com/api` or `http://localhost:3000/api`).
- **Complete Schema Export:** Exported as `swagger.json` inside the backend codebase for instant client code-generation or Postman import.
- **Core Resource Routes:**
  - `/user/*` — Profile management, authentication, passwords, and verification.
  - `/agent/*` — Agent registration, onboarding, and document uploads.
  - `/marketproduct/*` — Product catalog, listings, categories, and shopping cart.
  - `/admin/*` — Operations, approvals, moderation, metrics, and user management.
  - `/contact/*` — Inquiries, customer support, and issue tickets.

## 4. WEBHOOKS, EVENTS & SANDBOX ENVIRONMENTS

- **Sandbox / Staging Environments:**
  - Staging API Base URL: `https://api-dev.mejarc.com` (or local environment at `http://localhost:3000`).
  - Dedicated staging PostgreSQL database and test mail server (Mailtrap/SMTP sandbox) for OTP testing.
- **Webhooks & Asynchronous Events:**
  - External payment providers (Paystack, Flutterwave, Stripe) and KYC verification services deliver webhooks to designated webhook endpoints.
  - All incoming webhooks require cryptographic signature verification (e.g., `x-paystack-signature` or HMAC-SHA256 digest).